

习题

1. 给出以下概念的解释说明。

过程调用	调用约定	非静态局部变量	现场信息
栈 (stack)	ABI 规范	叶子过程	当前栈帧
调用者保存寄存器	被调用者保存寄存器	帧指针寄存器	按值传递参数
按地址传递参数	嵌套调用	递归调用	栈溢出
缓冲区溢出	缓冲区溢出攻击	栈随机化	金丝雀值

2. 简单回答下列问题。

- (1) 按值传递参数和按地址传递参数两种方式有哪些不同点?
- (2) 为什么在递归深度较深时递归调用的时间开销和空间开销都会较大?
- (3) 对于 auto 型变量, 编译器如何分配其空间? 通常被分配在什么存储区? 对于 auto 型变量地址进行大小比较, 是否有意义? 为什么?
- (4) 为什么数据在存储器中最好按对齐方式存放?
- (5) 有哪几种防止缓冲区溢出攻击的基本方法?

3. 假设某个 C 语言函数 func() 的原型声明如下:

```
void func(int *xptr, int *yptr, int *zptr);
```

函数 func() 的过程体对应的机器级代码用 AT&T 汇编形式表示如下:

```
1 movl 8(%ebp), %eax
2 movl 12(%ebp), %ebx
3 movl 16(%ebp), %ecx
4 movl (%ebx), %edx
5 movl (%ecx), %esi
6 movl (%eax), %edi
7 movl %edi, (%ebx)
8 movl %edx, (%ecx)
9 movl %esi, (%eax)
```

回答下列问题或完成下列任务。

- (1) 上述机器级代码是在 IA-32 系统还是 x86-64 系统中生成的? 为什么?
- (2) 在过程体开始时, 三个入口参数对应实参所存放的存储单元地址是什么? (提示: 当前栈帧底部由帧指针寄存器 EBP 指示。)
- (3) 根据上述机器级代码写出函数 func() 的 C 语言代码。

4. 假设函数 operate() 的部分 C 语言代码如下:

```
1 int operate(int x, int y, int z, int k) {
2     int v = _____ ;
3     return v;
4 }
```

以下 IA-32 汇编代码用来实现第 2 行语句的功能:

```
1 movl 12(%ebp), %ecx
2 sall $8, %ecx
```



```

3  movl    8(%ebp), %eax
4  movl    20(%ebp), %edx
5  imull   %edx, %eax
6  movl    16(%ebp), %edx
7  andl    $65520, %edx
8  addl    %ecx, %edx
9  subl    %edx, %eax

```

回答下列问题或完成下列任务。

(1) 写出每条汇编指令的注释，并填写 operate() 函数中缺失的部分。

(2) 给出函数 operate() 对应的 x86-64 汇编代码，并和 IA-32 汇编代码进行性能比较。

⑤ 假设函数 product() 的 C 语言代码如下，其中 num_type 是用 typedef 声明的数据类型。

```

1 void product(num_type *d, unsigned x, num_type y) {
2     *d = x*y;
3 }

```

函数 product() 的过程体对应的 IA-32 汇编代码如下：

```

1  movl    12(%ebp), %eax
2  movl    20(%ebp), %ecx
3  imull   %eax, %ecx
4  mull    16(%ebp)
5  leal    (%ecx, %edx), %edx
6  movl    8(%ebp), %ecx
7  movl    %eax, (%ecx)
8  movl    %edx, 4(%ecx)

```

给出上述每条汇编指令的注释，并说明 num_type 是什么类型。

⑥ 已知函数 comp() 的 C 语言代码及其过程体对应的汇编代码如图 4.28 所示。回答下列问题或完成下列任务。

(1) 图中给出的是 IA-32 还是 x86-64 对应的汇编代码？为什么？

(2) 给出每条汇编指令的注释，并说明为什么 C 语言代码中只有一个 if 语句而汇编代码中有两条条件跳转指令。

1	void comp(char x, int *p) {	1	testq %rsi, %rsi
2		2	je .L1
3	if (p && x<0)	3	testb \$0x80, %dil
4	*p += x;	4	jns .L1
5	}	5	movsbl %dil, %edi
		6	addl %edi, (%rsi)
		7	.L1:

图 4.28 题 6 图

⑦ 已知函数 func() 的 C 语言代码框架及其过程体对应的 IA-32 汇编代码如图 4.29 所示，根据对应的汇编代码填写 C 语言代码中缺失的表达式。

⑧ 已知函数 do_loop() 的 C 语言代码如下：

```

1 short do_loop(short x, short y, short k) {
2     do {

```



```

3      x*=(y%k) ;
4      k--;
5      } while ((k>0) && (y>k));
6      return x;
7  }

```

```

1 int func(int x, int y) {
2
3     int z = _____;
4     if (_____) {
5         if (_____)
6             z = _____;
7         else
8             z = _____;
9     } else if (_____)
10        z = _____;
11    return z;
12}

```

```

1    movl    8(%ebp), %eax
2    movl    12(%ebp), %edx
3    cmpl    $-100, %eax
4    jg      .L1
5    cmpl    %eax, %edx
6    jle     .L2
7    addl    %edx, %eax
8    jmp     .L3
9 .L2:
10   subl    %edx, %eax
11   jmp     .L3
12 .L1:
13   cmpl    $16, %eax
14   jl      .L4
15   andl    %edx, %eax
16   jmp     .L3
17 .L4:
18   imull   %edx, %eax
19 .L3:

```

图 4.29 题 7 图

函数 do_loop() 的过程体对应的 IA-32 汇编代码如下：

```

1    movw    8(%ebp), %bx
2    movw    12(%ebp), %si
3    movw    16(%ebp), %cx
4 .L1:
5    movw    %si, %dx
6    movw    %dx, %ax
7    sarw    $15, %dx
8    idiv    %cx
9    imulw    %dx, %bx
10   decw    %cx
11   testw    %cx, %cx
12   jle     .L2
13   cmpw    %cx, %si
14   jg      .L1
15 .L2:
16   movswl   %bx, %eax

```

回答下列问题或完成下列任务。

- (1) 给每条汇编指令添加注释，并说明每条指令执行后，目的寄存器中存放的是什么内容？
- (2) 上述函数过程体中用到了哪些被调用者保存寄存器和哪些调用者保存寄存器？在该函数过程体前面的准备阶段，哪些寄存器必须保存到栈中？
- (3) 为什么第 7 行中的 DX 寄存器需要算术右移 15 位？
- (4) 给出 do_loop() 函数对应的 x86-64 汇编代码。

- ⑨ 已知函数 f1() 的 C 语言代码框架及对应的 x86-64 汇编代码如图 4.30 所示，根据汇编代码填写 C 语言代码中缺失部分，并说明函数 f1() 的功能。

```

1 int f1(unsigned x) {
2
3     int y = 0 ;
4     while (_____) {
5         _____ ;
6     }
7     return _____ ;
8 }

```

```

1  movl    $0, %eax
2  testl   %edi, %edi
4  je      .L1
5  .L2:
6  xorl    %edi, %eax
7  shrl    $1, %edi
8  jne     .L2
9  .L1:
10 andl    $1, %eax

```

图 4.30 题 9 图

- ⑩ 已知函数 sw() 的 C 语言代码框架如下：

```

int sw(int x) {
    int v=0;
    switch (x) {
        /* switch 语句中的处理部分省略 */
    }
    return v;
}

```

函数 sw() 过程体中开始部分的 IA-32 汇编代码以及跳转表如图 4.31 所示。

```

1  movl    8(%ebp), %eax
2  addl    $3, %eax
3  cmpl    $7, %eax
4  ja      .L7
5  jmp     *.L8( , %eax, 4)
6  .L7:
7  ...
8  ...

```

```

1  .L8:
2      .long    .L7
3      .long    .L2
4      .long    .L2
5      .long    .L3
6      .long    .L4
7      .long    .L5
8      .long    .L7
9      .long    .L6

```

图 4.31 题 10 图

回答下列问题。

- (1) 函数 sw() 中的 switch 语句处理部分标号的取值情况如何？
- (2) 标号的取值在什么情况下执行 default 分支？哪些标号的取值会执行同一个 case 分支？

- ⑪ 已知 C 语言函数 test() 的入口参数有 a、b、c 和 p，过程体代码如下：

```

*p = a;
return b*c;

```

函数 test() 过程体对应的 IA-32 汇编代码如下：

```

1  movl    20(%ebp), %edx
2  movsbw  8(%ebp), %ax
3  movw    %ax, (%edx)
4  movzwl  12(%ebp), %eax
5  movzwl  16(%ebp), %ecx

```



```
6  mull    %ecx
```

完成下列任务。

- (1) 写出函数 test() 的原型，以给出返回参数的类型以及入口参数 a、b、c 和 p 的类型和顺序。
- (2) 写出对应的 x86-64 汇编代码。

12. 已知函数 funct() 的 C 语言代码如下：

```
1  #include <stdio.h>
2  int funct(void) {
3      int x, y;
4      scanf("%d %d", &x, &y);
5      return x-y;
6  }
```

函数 funct() 对应的 IA-32 汇编代码如下：

```
1  funct:
2      pushl    %ebp
3      movl     %esp, %ebp
4      subl     $40, %esp
5      leal     -8(%ebp), %eax
6      movl     %eax, 8(%esp)
7      leal     -4(%ebp), %eax
8      movl     %eax, 4(%esp)
9      movl     $.LC0, (%esp)      # 将指向字符串 "%d %d" 的指针入栈
10     call     scanf              # 假定 scanf 执行后 x=15, y=20
11     movl     -4(%ebp), %eax
12     subl     -8(%ebp), %eax
13     leave
14     ret
```

假设函数 funct() 开始执行时，R[esp]=0xbc00 0020，R[ebp]=0xbc00 0030，指向字符串 “%d %d” 的指针为 0x804 c000。回答下列问题或完成下列任务。

- (1) 执行第 3、10 和 13 行的指令后，寄存器 EBP 中的内容分别是什么？
- (2) 执行第 3、10 和 13 行的指令后，寄存器 ESP 中的内容分别是什么？
- (3) 局部变量 x 和 y 所在存储单元的地址分别是什么？
- (4) 画出执行第 10 行指令后 funct 的栈帧，给出栈帧中的内容及其地址。

13. 已知递归函数 refunc() 的 C 语言代码框架如下：

```
1  int refunc(unsigned x) {
2      if ( _____ )
3          return _____ ;
4      unsigned nx = _____ ;
5      int rv = refunc(nx) ;
6      return _____ ;
7  }
```

上述递归函数过程体对应的 IA-32 汇编代码如下：

```
1  movl     8(%ebp), %ebx
2  movl     $0, %eax
```



```
3    testl    %ebx, %ebx
4    je       .L2
5    movl     %ebx, %eax
6    shrl     $1, %eax
7    movl     %eax, (%esp)
8    call     refunc
9    movl     %ebx, %edx
10   andl     $1, %edx
11   leal     (%edx, %eax), %eax
12 .L2:
```

根据对应的汇编代码填写 C 语言代码中缺失的部分，并说明函数的功能。

- 14 针对 IA-32 和 x86-64 两种系统，填写表 4.4，说明每个数组元素的大小、整个数组的大小以及第 i 个元素的地址。

表 4.4 题 14 表

数组	元素大小 /B	数组大小 /B	起始地址	元素 i 的地址
int A[10]			&A[0]	
long B[100]			&B[0]	
short*C[5]			&C[0]	
short**D[6]			&D[0]	
long double E[10]			&E[0]	
long double *F[10]			&F[0]	

- 15 假设在 x86-64 系统中，short 型数组 S 的首地址 AS 和数组下标（索引）变量 i 分别存放在寄存器 RDX 和 RCX 中，表 4.5 给出的表达式的结果存放在 RAX 或 AX 中，仿照例子填写表 4.5，说明表达式的类型、值和相应的汇编代码。

表 4.5 题 15 表

表达式	类型	值	汇编代码
S			
S+i-3			
S[i]	short	M[AS+2*i]	movw (%rdx, %rcx, 2), %ax
&S[10]			
&S[i+2]	short *	AS+2*i+4	leaq 4(%rdx, %rcx, 2), %rax
&S[i]-S			
S[4*i+4]			
*(S+i-2)			

- 16 假设函数 sumij() 的 C 语言代码如下，其中，M 和 N 是用 #define 声明的常数。

```
1 int a[M][N], b[N][M];
2
3 int sumij(int i, int j) {
4     return a[i][j] + b[j][i];
5 }
```

已知函数 sumij() 的 IA-32 过程体对应的汇编代码如下：


```

1 movl    8(%ebp), %ecx
2 movl    12(%ebp), %edx
3 leal    (, %ecx, 8), %eax
4 subl    %ecx, %eax
5 addl    %edx, %eax
6 leal    (%edx, %edx, 4), %edx
7 addl    %ecx, %edx
8 movl    a(, %eax, 4), %eax      # a 表示数组 a 的首地址
9 addl    b(, %edx, 4), %eax      # b 表示数组 b 的首地址

```

根据上述汇编代码，确定 M 和 N 的值。

17. 假设函数 `st_ele()` 的 C 语言代码如下，其中，L、M 和 N 是用 `#define` 声明的常数。

```

1 int a[L][M][N];
2
3 int st_ele(int i, int j, int k, int *dst) {
4     *dst = a[i][j][k];
5     return sizeof(a);
6 }

```

已知函数 `st_ele()` 的过程体对应的 IA-32 汇编代码如下：

```

1 movl    8(%ebp), %ecx
2 movl    12(%ebp), %edx
3 leal    (%edx,%edx, 8), %edx
4 movl    %ecx, %eax
5 sall    $6, %eax
6 subl    %ecx, %eax
7 addl    %eax, %edx
8 addl    16(%ebp), %edx
9 movl    a(, %edx, 4), %eax      # a 表示数组 a 的首地址
10 movl    20(%ebp), %edx
11 movl    %eax, (%edx)
12 movl    $4536, %eax

```

根据上述汇编代码确定 L、M 和 N 的值，并写出函数 `st_ele()` 对应的 x86-64 汇编代码。

18. 假设函数 `trans_matrix()` 的 C 语言代码如下，其中，M 是用 `#define` 声明的常数。

```

1 void trans_matrix(int a[M][M]) {
2     int i, j, t;
3     for (i = 0; i < M; i++)
4         for (j = 0; j < M; j++) {
5             t = a[i][j];
6             a[i][j] = a[j][i];
7             a[j][i] = t;
8         }
9 }

```

已知采用优化编译（选项 `-O2`）后函数 `trans_matrix()` 的内循环对应的 IA-32 汇编代码如下：

```

1 .L2:
2     movl    (%ebx), %eax
3     movl    (%esi, %ecx, 4), %edx

```



```

4  movl    %eax, (%esi, %ecx, 4)
5  addl    $1, %ecx
6  movl    %edx, (%ebx)
7  addl    $76, %ebx
8  cmpl    %edi, %ecx
9  jl      .L2

```

根据上述汇编代码，回答下列问题或完成下列任务。

(1) M 的值是多少？常数 M 和变量 j 分别存放在哪个寄存器中？

(2) 写出上述优化汇编代码对应的函数 trans_matrix() 的 C 代码。

19. 假设结构体类型 node 的定义、函数 np_init() 部分 C 语言代码及对应的 IA-32 部分汇编代码如图 4.32 所示。

```

struct node {
    int *p;
    struct {
        int x;
        int y;
    } s;
    struct node *next;
};

```

```

void np_init(struct node *np)
{
    np->s.x = _____;
    np->p = _____;
    np->next = _____;
}

```

```

movl    8(%ebp), %eax
movl    8(%eax), %edx
movl    %edx, 4(%eax)
leal    4(%eax), %edx
movl    %edx, (%eax)
movl    %eax, 12(%eax)

```

图 4.32 题 19 图

回答下列问题或完成下列任务。

(1) 结构体 node 所需存储空间为多少字节？成员 p、s.x、s.y 和 next 的偏移地址分别为多少？

(2) 根据汇编代码填写 np_init() 中缺失的表达式。

(3) 写出图 4.32 中 IA-32 汇编代码对应的 x86-64 汇编代码。

20. 假设联合体类型 utype 的定义如下：

```

typedef union {
    struct {
        int x;
        short y;
        short z;
    } s1;
    struct {
        short a[2];
        int b;
        char *p;
    } s2;
} utype;

```

若存在具有如下形式的一组函数：

```

void getvalue(utype *uptr, TYPE *dst) {
    *dst = EXPR;
}

```

该组函数用于计算不同表达式 EXPR 的值，返回值的数据类型根据表达式的类型确定。假设函数 getvalue() 的入口参数 uptr 和 dst 分别被装入寄存器 EAX 和 EDX 中，仿照例子填写表 4.6，说明

在不同表达式下的 TYPE 类型以及表达式对应的 IA-32 汇编指令序列 (要求尽量只使用 EAX 和 EDX, 不够用时再使用 ECX)。

表 4.6 题 20 表

表达式 EXPR	TYPE 类型	汇编指令序列
uptr->s1.x	int	movl (%eax), %eax movl %eax, (%edx)
uptr->s1.y		
&uptr->s1.z		
uptr->s2.a		
uptr->s2.a[uptr->s2.b]		
*uptr->s2.p		

21. 分别给出在 IA-32+Linux、x86-64+Linux 平台下, 下列各个结构体类型中每个成员的偏移量、结构体总大小以及结构体起始位置的对齐要求。

- (1) struct S1 {short s; char c; int i; char d;};
- (2) struct S2 {int i; short s; char c; char d;};
- (3) struct S3 {char c; short s; int i; char d;};
- (4) struct S4 {short s[3]; char c;};
- (5) struct S5 {char c[3]; short *s; int i; char d; double e;};
- (6) struct S6 {struct S1 c[3]; struct S2 *s; char d;};

22. 以下是结构体 test 的声明:

```
struct {
    char      c;
    double    d;
    int       i;
    short     s;
    char      *p;
    long      l;
    long long g;
    void      *v;
} test;
```

假设在 Windows 平台上编译, 则这个结构体中每个成员的偏移量是多少? 结构体总大小为多少字节? 如何调整成员的先后顺序使结构体所占空间最小?

23. 图 4.33 给出了函数 getline() 存在漏洞和问题的 C 语言代码实现, 右边是其对应的 IA-32 反汇编部分结果。

```
char *getline() {
    char buf[8];
    char *result;
    gets(buf);
    result=malloc(strlen
        (buf));
    strcpy(result, buf);
    return result;
}
```

```
1 0804840c <getline>:
2   804840c: 55          push    %ebp
3   804840d: 89 e5       mov     %esp, %ebp
4   804840f: 83 ec 28    sub     $0x28, %esp
5   8048412: 89 5d f4    mov     %ebx, -0xc(%ebp)
6   8048415: 89 75 f8    mov     %esi, -0x8(%ebp)
7   8048418: 89 7d fc    mov     %edi, -0x4(%ebp)
8   804841b: 8d 75 ec    lea     -0x14(%ebp), %esi
9   804841e: 89 34 24    mov     %esi, (%esp)
10  8048421: e8 a3 ff ff call    80483c9 <gets>
```

图 4.33 题 23 图